

1. memory_and_test_shell.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 32
#define BUF_SIZE 1024

// Dynamically allocate prompt string to demonstrate dynamic memory tracking & clean release
char *create_dynamic_prompt(const char *shell_name) {
    char *prompt = malloc(strlen(shell_name) + 4);
    if (!prompt) {
        perror("malloc failed");
        exit(EXIT_FAILURE);
    }
    sprintf(prompt, "%s > ", shell_name);
    return prompt;
}

// Trim leading and trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    // Dynamically allocate argument array to verify leak-free execution
    char **args = calloc(MAX_ARGS, sizeof(char *));
```

```

    if (!args) {
        perror("calloc failed");
        return;
    }

    int arg_count = 0;
    char *token = strtok(line, " \\t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \\t");
    }
    args[arg_count] = NULL;

    if (arg_count > 0) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
        } else if (pid == 0) {
            execvp(args[0], args);
            perror("Command execution failed");
            free(args); // Clean release in child
            exit(EXIT_FAILURE);
        } else {
            int status;
            waitpid(pid, &status, 0);
        }
    }

    // Cleanly release allocated memory resources in parent
    free(args);
}

int main() {
    char input_buf[BUF_SIZE];
    char *prompt = create_dynamic_prompt("ValgrindShell");

```

```

// Enable automated non-interactive pipeline testing
int is_interactive = isatty(STDIN_FILENO);

while (1) {
    if (is_interactive) {
        printf("%s", prompt);
        fflush(stdout);
    }

    if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

    input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

    if (strcmp(input_buf, "exit") == 0) break;
    if (strlen(input_buf) == 0) continue;

    execute_command(input_buf);
}

// Explicitly free dynamic resources before exit to pass Valgrind checks
free(prompt);
return 0;
}

```

2. run_tests.sh

```

GNU nano 8.7.1 run_tests.sh
#!/bin/bash

EXECUTABLE="./memory_and_test_shell"
REPORT_FILE="test_report.txt"

echo "===== " > $REPORT_FILE
echo " AUTOMATED TEST EXECUTION & VERIFICATION " >> $REPORT_FILE
echo "===== " >> $REPORT_FILE
echo "Date: $(date)" >> $REPORT_FILE
echo "" >> $REPORT_FILE

# Test Case 1: Simple Command Execution
echo "Running Test Case 1: Simple Command..."
echo "echo 'Test Case 1 Passed'" | $EXECUTABLE > test1_out.tmp 2>&1
if grep -q "Test Case 1 Passed" test1_out.tmp; then
    echo "[PASS] Test 1: Simple Echo Execution" >> $REPORT_FILE
else
    echo "[FAIL] Test 1: Simple Echo Execution" >> $REPORT_FILE
fi

# Test Case 2: Multi-Argument Command
echo "Running Test Case 2: Multi-Argument Output..."
echo "uname -s" | $EXECUTABLE > test2_out.tmp 2>&1
if [ -s test2_out.tmp ]; then
    echo "[PASS] Test 2: System Command Output Verified" >> $REPORT_FILE
else
    echo "[FAIL] Test 2: System Command Output Empty" >> $REPORT_FILE
fi

# Test Case 3: Exit Cleanly
echo "Running Test Case 3: Exit Processing..."
echo "exit" | $EXECUTABLE > test3_out.tmp 2>&1
if [ $? -eq 0 ]; then
    echo "[PASS] Test 3: Shell Exit Behavior Clean" >> $REPORT_FILE
else
    echo "[FAIL] Test 3: Shell Exit Non-Zero" >> $REPORT_FILE
fi

rm -f test1_out.tmp test2_out.tmp test3_out.tmp

echo "" >> $REPORT_FILE
echo "Automated test suite completed. Results appended to $REPORT_FILE"
cat $REPORT_FILE

```

OUTPUT :

```

abhis@MIKHAIL-37:~$ nano memory_and_test_shell.c
abhis@MIKHAIL-37:~$ gcc -g -Wall memory_and_test_shell.c -o memory_and_test_shell
abhis@MIKHAIL-37:~$ nano run_tests.sh
abhis@MIKHAIL-37:~$ chmod +x run_tests.sh
abhis@MIKHAIL-37:~$ ./run_tests.sh
Running Test Case 1: Simple Command...
Running Test Case 2: Multi-Argument Output...
Running Test Case 3: Exit Processing...
Automated test suite completed. Results appended to test_report.txt
=====
AUTOMATED TEST EXECUTION & VERIFICATION
=====
Date: Thu Oct 1 05:42:19 UTC 2026

[PASS] Test 1: Simple Echo Execution
[PASS] Test 2: System Command Output Verified
[PASS] Test 3: Shell Exit Behavior Clean

abhis@MIKHAIL-37:~$ valgrind --leak-check=full --show-leak-kinds=all ./memory_and_test_shell
==1505== Memcheck, a memory error detector
==1505== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1505== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1505== Command: ./memory_and_test_shell
==1505==
ValgrindShell > echo "Valgrind Memory Test"
"Valgrind Memory Test"

```

```

"Valgrind Memory Test"
ValgrindShell > ls
2.c          cow_demo    history_shell.c  memory_and_test_shell.c  path_waitpid.c  shell_history.c  vars_dispatch
OS           cow_demo.c  integrated_shell  memory_layout            pgm.c           shell_state.txt  vars_dispatch.c
append_stderr  er         integrated_shell.c  memory_layout.c         pgm.c           signal_bg_recovery  word_count.txt
append_stderr.c  er.c       io_redirection    memory_test              process.c        signal_bg_recovery.c  ww.c
background_jobs  error.log  io_redirection.c  memory_test.c           process.c        signal_handler      zx.c
background_jobs.c  error_summary.txt  job_listing_fg    merged.log              program.c        signal_handler.c    zx
bh              executor   job_listing_fg.c  multi_pipe              program.c        skill2              zx.c
bh.c            executor.c  job_suspension_pgid  multi_pipe.c           quote_parser.c   skill2.c           stderr.log
cd_dispatch      export_state  job_suspension_pgid.c  output.txt            quotes_parser    stdout.log         test_report.txt
cd_dispatch.c    export_state.c  ll                 output_users.txt       quotes_parser.c  test_report.txt   tokenizer
combined.log     history_pipeline  ll.c              parser_test             parser_test.c   shell_errors.log   tokenizer.c
combined_complex  history_pipeline.c  log.txt          parser_test.c           path_waitpid    shell_history
combined_complex.c  history_shell  memory_and_test_shell
ValgrindShell > exit
==1505==
==1505== HEAP SUMMARY:
==1505==   in use at exit: 0 bytes in 0 blocks
==1505==   total heap usage: 5 allocs, 5 frees, 2,577 bytes allocated
==1505==
==1505== All heap blocks were freed -- no leaks are possible
==1505==
==1505== For lists of detected and suppressed errors, rerun with: -s
==1505== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```